

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO
THỰC TẬP TỐT NGHIỆP
CỦA NHÂN

Sinh viên thực hiện	Nguyễn Kim Trường Giang
Mã sinh viên	223630679
Lớp	Khoa học máy tính
Khóa	K63
Ngành đào tạo	Khoa học máy tính
Giảng viên hướng dẫn	TS. Nguyễn Quốc Tuấn
CBHD tại đơn vị TT	Nguyễn Dũng Tiến
Thời gian thực tập	12/01/2026 - 06/03/2026

Hà Nội – 2026

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO

**THỰC TẬP TỐT NGHIỆP
CỦA NHÂN**

Sinh viên thực hiện	Nguyễn Kim Trường Giang
Mã sinh viên	223630679
Lớp	Khoa học máy tính
Khóa	63
Ngành đào tạo	Khoa học máy tính
Giảng viên hướng dẫn	TS. Nguyễn Quốc Tuấn
CBHD tại đơn vị TT	Nguyễn Dũng Tiến
Đơn vị thực tập:	Công ty TNHH phần mềm FPT
Thời gian thực tập	12/01/2026 - 06/03/2026

Hà Nội – 2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc đến Quý Thầy/Cô Khoa Công nghệ thông tin, Trường đại học Giao Thông Vận Tải đã tạo điều kiện cho em được tham gia kỳ thực tập này. Đây là cơ hội quý báu giúp em có thể vận dụng những kiến thức đã học vào thực tế và tích lũy thêm nhiều kinh nghiệm bổ ích cho bản thân.

Em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Nguyễn Quốc Tuấn – Giảng viên hướng dẫn, người đã tận tình chỉ bảo, định hướng và hỗ trợ em trong suốt quá trình thực tập cũng như hoàn thành báo cáo này. Những góp ý chuyên môn và sự hướng dẫn tận tâm của thầy là nền tảng quan trọng giúp em hoàn thành tốt nhiệm vụ được giao.

Em cũng xin chân thành cảm ơn Ban lãnh đạo cùng toàn thể Anh/Chị tại FPT Software đã tạo điều kiện thuận lợi, cung cấp môi trường làm việc chuyên nghiệp và hỗ trợ em trong quá trình thực tập. Sự hướng dẫn, chia sẻ kinh nghiệm thực tế của các Anh/Chị đã giúp em hiểu rõ hơn về quy trình làm việc, kỹ năng chuyên môn cũng như tác phong nghề nghiệp.

Do thời gian thực tập và kiến thức còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của Quý Thầy/Cô và đơn vị để em có thể hoàn thiện hơn trong tương lai.

Em xin chân thành cảm ơn!

MỤC LỤC

DANH MỤC HÌNH VẼ	6
DANH MỤC BẢNG BIỂU	6
DANH MỤC TỪ VIẾT TẮT	7
1. GIỚI THIỆU CHUNG VỀ ĐƠN VỊ THỰC TẬP	8
1.1. Thông tin chung về đơn vị thực tập	8
1.2. Chức năng, nhiệm vụ của bộ phận thực tập	8
1.3. Môi trường làm việc và quy trình công tác	8
2. NỘI DUNG THỰC TẬP	9
2.1. Mục tiêu và yêu cầu của đợt thực tập	9
2.2. Các công việc đã thực hiện	9
2.3. Công nghệ, công cụ và kỹ thuật sử dụng	9
3. KIẾN THỨC TÌM HIỂU	10
3.1. Python – PydanticAI – FastAPI – PyGithub	10
3.1.1. Python	10
3.1.2. PydanticAI	11
3.1.3. FastAPI	11
3.1.4. PyGithub	12
3.2. TypeScript – NextJS	13
3.2.1. TypeScript	13
3.2.2. NextJS	13
3.2.3. Inngest – Nền tảng Điều phối Sự kiện	14
3.3. Tìm hiểu về bài toán tự động hóa đánh giá mã nguồn	15
3.3.1. Kỹ thuật truyền thống	15
3.3.2. Kỹ thuật trí tuệ nhân tạo tạo sinh dựa trên các mô hình ngôn ngữ lớn ...	16
3.4. Ứng dụng RAG và Vector Database vào bài toán đánh giá mã nguồn	17
3.4.1. Giới thiệu về Retrieval-Augmented Generation (RAG)	17
3.4.2. Giới thiệu về Cơ sở dữ liệu Vector (Vector Database) - Pinecone	18
3.4.3. Thiết lập môi trường lập trình	21
3.4.4. Tải và cấu trúc luồng dữ liệu	22
3.4.5. Xây dựng luồng xử lý RAG	22
3.4.6. Triển khai API và xây dựng UI mô phỏng	23
4. NHẬN XÉT VÀ ĐỊNH HƯỚNG	25

4.1. Nhận xét chung về đợt thực tập	25
4.2. Bài học kinh nghiệm rút ra	25
4.3. Định hướng nghề nghiệp và học tập sau thực tập	25
TÀI LIỆU THAM KHẢO	26

DANH MỤC HÌNH VẼ

Hình 1: Sơ đồ minh họa nguyên lý hoạt động của kiến trúc RAG	16
Hình 2: Nguyên lý vận hành của Pinecone (một loại vector database).....	18

DANH MỤC BẢNG BIỂU

Bảng 1: Bảng so sánh hai kiến trúc RAG và kỹ thuật Fine-tuning	18
Bảng 2: So sánh các loại database	20

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Giải thích ý nghĩa
JLPT	Japanese-Language Proficiency Test	Kì thi đánh giá năng lực tiếng Nhật
FSoft	FPT Software	Công ty TNHH Phần mềm FPT
FSA	FPT Software Academy	Viện đào tạo thuộc FPT Software

1. GIỚI THIỆU CHUNG VỀ ĐƠN VỊ THỰC TẬP

1.1. Thông tin chung về đơn vị thực tập

Tên đơn vị: FPT Software (FSoft)

Lĩnh vực hoạt động: Kỹ sư cầu nối Việt Nhật BrSE (Bridge Software Engineer)

Cơ cấu tổ chức: Hoạt động theo mô hình là một công ty công nghệ lớn bao gồm các phòng ban, đội ngũ kỹ thuật, bộ phận đào tạo, bộ phận nhân sự và đội ngũ hỗ trợ.

Quy mô và định hướng phát triển: Là một trong những công ty công nghệ lớn tại Việt Nam. Định hướng phát triển tại nhiều quốc gia, có các mạng lưới đối tác tại 30 quốc gia và vùng lãnh thổ trên thế giới. Về nhân sự, FPT Software có hơn 32,000 kỹ sư và chuyên gia công nghệ

1.2. Chức năng, nhiệm vụ của bộ phận thực tập

Tham gia vào đội ngũ thực tập sinh theo hướng kỹ sư cầu nối BrSE giữa hai nước Việt - Nhật.

Nhiệm vụ chính: Tiếp nhận theo lộ trình được bộ phận đào tạo trực thuộc FPT thiết kế sẵn bao gồm hai nhiệm vụ cốt lõi: tham gia học tập tiếng Nhật và đào tạo chuyên sâu về 2 ngôn ngữ lập trình Java, C#

1.3. Môi trường làm việc và quy trình công tác

Môi trường làm việc năng động, linh hoạt, khuyến khích sáng tạo và khả năng tự học.

Quy trình công tác: Sử dụng các thiết bị, tài nguyên, công cụ được công ty cấp cho sinh viên thực tập để phục vụ và hoàn thành các nhiệm vụ tại cơ sở thực tập.

2. NỘI DUNG THỰC TẬP

2.1. Mục tiêu và yêu cầu của đợt thực tập

Mục tiêu đầu ra: Đạt được trình độ giao tiếp và đọc hiểu các văn bản tiếng Nhật ở trình độ JLPT N3, có khả năng biên phiên dịch các tài liệu chuyển đổi giữa hai ngôn ngữ Việt – Nhật trong quá trình học tập và làm việc. Ngoài ra, đạt yêu cầu để thông qua các bài kiểm tra về khả năng lập trình bằng Java và C# sau quá trình được đào tạo bởi đội ngũ hỗ trợ của công ty.

2.2. Các công việc đã thực hiện

Tham gia chương trình đào tạo kỹ sư cầu nối theo lộ trình được thiết kế sẵn bởi bộ phận đào tạo FSA trực thuộc FPT Software

Hoàn thành các bài thi khảo sát đầu ra của mỗi khóa học trong danh sách khóa học của chương trình đào tạo.

2.3. Công nghệ, công cụ và kỹ thuật sử dụng

Ngôn ngữ lập trình: Java, C#

Framework/Nền tảng: Spring Boot, ASP.Net Core

Công cụ hỗ trợ: Visual Code, IntelliJ IDEA, Git, Docker.

3. KIẾN THỨC TÌM HIỂU

3.1. Python – PydanticAI – FastAPI – PyGithub

3.1.1. Python

Python là một ngôn ngữ lập trình bậc cao, thông dịch, hướng đối tượng và sử dụng đa mục đích như ứng dụng web, phát triển phần mềm, khoa học dữ liệu và học máy (Machine Learning)

Python được tạo ra bởi Guido van Rossum và phát hành lần đầu tiên vào năm 1991.

Các nhà phát triển sử dụng Python vì nó hiệu quả, cú pháp đơn giản, dễ học, dễ đọc và có thể chạy trên nhiều nền tảng khác nhau. Phần mềm Python được tải xuống miễn phí, tích hợp tốt với tất cả các loại hệ thống và tăng tốc độ phát triển. [4]

Các đặc điểm nổi bật của Python:

- **Cú pháp đơn giản, dễ đọc:** Python có cú pháp ngắn gọn, rõ ràng, dễ đọc, và gần giống với ngôn ngữ tự nhiên (tiếng Anh), giúp người mới có thể học một cách dễ dàng mà không cần kiến thức sâu rộng về lập trình
- **Đa nền tảng (Multi-platform):** Ta có thể viết mã Python trên một nền tảng và chạy trên nền tảng khác mà không cần thay đổi mã nguồn. [4]
- **Đa phong cách lập trình:** Python hỗ trợ nhiều phong cách lập trình khác nhau, bao gồm:
 - Lập trình thủ tục (Procedure Programming): Sử dụng các hàm và câu lệnh tuần tự
 - Lập trình hướng đối tượng (OOP – Object Oriented Programming): Sử dụng lớp (Class) và đối tượng (Object)
 - Lập trình hàm (Functional Programming) [4]

Thư viện và framework phong phú: Python có hệ sinh thái thư viện và framework phong phú, giúp giải quyết nhiều bài toán lập trình:

- Khoa học dữ liệu: Numpy, Pandas, Matplotlib, Seaborn
- Học máy và AI: Tensorflow, Pytorch, Scikit-learn
- Phát triển web: Django, Flask, FastAPI
- Tự động hóa: Selenium, BeautifulSoup
- AI Agent & LLM: PydanticAI, LangChain
- Tích hợp hệ thống: PyGithub, Requests

3.1.2. PydanticAI

PydanticAI là một Framework Python hiện đại dành cho việc xây dựng các ứng dụng AI Agentic (tác nhân AI), được phát triển bởi đội ngũ tạo ra thư viện Pydantic nổi tiếng.

Khác với các framework AI truyền thống, PydanticAI tập trung vào việc đảm bảo dữ liệu đầu ra từ mô hình ngôn ngữ lớn (LLM) luôn có cấu trúc chính xác, an toàn về kiểu dữ liệu (Type-safe) và dễ dàng kiểm soát. [3]

Các đặc điểm chính của PydanticAI:

- **Đảm bảo cấu trúc dữ liệu (Structured Outputs):** PydanticAI ép buộc các mô hình AI trả về dữ liệu dưới dạng JSON hoặc Object đã định nghĩa trước, thay vì văn bản tự do, giúp việc tích hợp vào hệ thống phần mềm trở nên tin cậy tuyệt đối.
- **Type-safe & Validation:** Tận dụng sức mạnh của Pydantic để kiểm tra tính hợp lệ của dữ liệu ngay khi AI sinh ra, giảm thiểu lỗi logic hoặc ảo giác (hallucination). [3]
- **Cơ chế Dependency Injection:** Cung cấp cách thức quản lý các phụ thuộc (dependencies) linh hoạt, giúp code gọn gàng và dễ bảo trì.
- **Hỗ trợ đa mô hình:** Tương thích tốt với nhiều nhà cung cấp LLM khác nhau như OpenAI, Google Gemini, Anthropic hay Grok. [3]

Các thành phần chính của PydanticAI:

- **Agent:** Đơn vị thực thi chính, chịu trách nhiệm nhận yêu cầu, gọi công cụ (Tools) và trả về kết quả.
- **Tools:** Các hàm chức năng mà Agent có thể gọi để thực hiện tác vụ thực tế (ví dụ: lấy thông tin GitHub, tra cứu database).
- **RunContext:** Ngữ cảnh chạy, giúp truyền tải dữ liệu và dependencies vào trong quá trình suy luận của Agent. [3]

3.1.3. FastAPI

FastAPI là một web framework hiện đại, hiệu năng cao để xây dựng các API bằng Python 3.8+, dựa trên các tiêu chuẩn Python type hints.

FastAPI được thiết kế để tối ưu hóa tốc độ phát triển và hiệu suất thực thi, là lựa chọn hàng đầu cho các hệ thống Backend phục vụ AI và Microservices. [4]

Các đặc điểm chính của FastAPI:

- **Hiệu năng cao:** Ngang ngửa với NodeJS và Go, nhờ sử dụng Starlette (cho web parts) và Pydantic (cho data parts).

- Xử lý bất đồng bộ (Asyncio): Hỗ trợ native async/await, giúp xử lý hàng ngàn kết nối đồng thời mà không bị tắc nghẽn, rất phù hợp cho các tác vụ I/O bound như gọi AI API hay Database. [4]
- Tự động tạo tài liệu: Tự động sinh tài liệu API tương tác (Swagger UI, ReDoc) dựa trên code, giúp việc kiểm thử và tích hợp trở nên dễ dàng. [4]
- Kiểm tra dữ liệu tự động: Sử dụng Pydantic để validate dữ liệu đầu vào và đầu ra, giảm thiểu tối đa các lỗi runtime.

Các thành phần chính của FastAPI:

- Path Operations: Các hàm xử lý request tại các đường dẫn (URL) cụ thể.
- Dependency Injection System: Hệ thống quản lý phụ thuộc mạnh mẽ, giúp tái sử dụng code logic (như xác thực, kết nối DB) một cách hiệu quả.
- Pydantic Models: Định nghĩa cấu trúc dữ liệu (Schema) cho Request Body và Response.

3.1.4. PyGithub

PyGithub là một thư viện Python mạnh mẽ dùng để tương tác với GitHub API v3. Nó cho phép quản lý các tài nguyên trên GitHub (Repositories, User Profiles, Organizations) thông qua các đối tượng Python (Object-Oriented).

Trong các dự án tích hợp phần mềm lưu trữ mã nguồn Github, PyGithub đóng vai trò cầu nối quan trọng để hệ thống tự động hóa các thao tác liên quan đến Github. [5]

Các đặc điểm chính của PyGithub:

- Hướng đối tượng: Thay vì gọi các URL API thô sơ, PyGithub trừu tượng hóa các tài nguyên thành các lớp (Class) như Repository, PullRequest, Commit, giúp code trực quan và dễ hiểu.
- Hỗ trợ đầy đủ tính năng: Bao phủ hầu hết các endpoint của GitHub API, từ việc lấy thông tin code, quản lý issue, đến việc tạo comment review. [5]
- Quản lý phân trang tự động: Tự động xử lý việc phân trang (Pagination) khi lấy danh sách dữ liệu lớn.

Các thành phần chính được sử dụng:

- Repository: Đại diện cho một kho chứa mã nguồn, cung cấp các phương thức để truy xuất file, commit. [5]
- Pull Request: Đại diện cho một yêu cầu gộp code, nơi diễn ra hoạt động review, lấy thông tin diff và đăng tải nhận xét. [5]
- Github Instance: Đối tượng khởi tạo kết nối, quản lý xác thực (Authentication) bằng Token.

3.2. TypeScript – NextJS

3.2.1. TypeScript

TypeScript là một ngôn ngữ lập trình mã nguồn mở phát triển và duy trì bởi Microsoft. TypeScript được giới thiệu để cải thiện và mở rộng ngôn ngữ JavaScript bằng cách bổ sung thêm tính năng các kiểu dữ liệu tĩnh và các cấu trúc lập trình hướng đối tượng.

Các đặc điểm chính của TypeScript:

- Kiểu dữ liệu tĩnh: TypeScript cho phép ta chỉ định rõ kiểu dữ liệu của một biến, tham số, hay giá trị trả về từ một hàm. Điều này giúp ta kiểm tra lỗi dữ liệu trong quá trình biên dịch, giúp phát hiện và sửa lỗi sớm hơn trong quá trình phát triển.
- Hướng đối tượng: TypeScript hỗ trợ các khái niệm lập trình hướng đối tượng như lớp, giao diện, kế thừa và đóng gói, giúp mã nguồn trở nên dễ đọc, tái sử dụng và duy trì tốt hơn. [6]
- Tooling – IntelliSense: TypeScript tích hợp chặt chẽ với các công cụ phát triển như Visual Studio Code, cung cấp IntelliSense mạnh mẽ để hỗ trợ lập trình viên hiểu và sử dụng API một cách thuận tiện.
- Duy trì cho dự án lớn: Với tính năng kiểu dữ liệu tĩnh, TypeScript thích hợp cho việc phát triển các dự án lớn.

3.2.2. NextJS

Next.js là một Framework React dành cho sản phẩm thực tế (Production-grade), được phát triển bởi Vercel. Nó mở rộng khả năng của React bằng cách cung cấp các giải pháp tối ưu hóa hiệu năng và trải nghiệm người dùng ngay từ đầu.

Trong dự án này, phiên bản Next.js 16 được sử dụng với kiến trúc App Router hiện đại. [7]

Các đặc điểm chính:

- Server Components (RSC): Cho phép render các component ngay trên server, giúp giảm dung lượng JavaScript gửi xuống client, tăng tốc độ tải trang ban đầu.
- Server Actions: Cho phép gọi các hàm server trực tiếp từ client mà không cần tạo API endpoint thủ công, đơn giản hóa việc xử lý dữ liệu form và mutation.
- Routing thông minh: Hệ thống File-system based routing (định tuyến dựa trên tệp tin) mạnh mẽ, hỗ trợ Nested Routes, Loading UI, và Error Handling một cách trực quan.

- Tích hợp API Routes: Next.js cho phép xây dựng các API Backend (Serverless Functions) ngay trong cùng một dự án, đóng vai trò là API Gateway để giao tiếp với các dịch vụ bên ngoài (như Python Backend).
- Tối ưu hóa (Optimization): Tự động tối ưu hóa hình ảnh, phông chữ và scripts để đạt điểm hiệu năng cao nhất trên các công cụ đánh giá (Core Web Vitals).

Các thành phần chính của Next.js (App Router):

- Page & Layout: Page là giao diện của một đường dẫn cụ thể, Layout là giao diện dùng chung cho nhiều trang (như Navbar, Sidebar), giúp giữ trạng thái khi điều hướng.
- Middleware: Cho phép chạy mã trước khi request hoàn tất, thường dùng để xác thực (Authentication) hoặc chuyển hướng người dùng.
- Data Fetching: Cơ chế lấy dữ liệu linh hoạt, hỗ trợ bộ nhớ đệm (Caching) và xác thực lại dữ liệu (Revalidation) để đảm bảo tính thời gian thực.
- Virtual DOM: Next.js (dựa trên ReactJS) sử dụng Virtual DOM, một bản sao nhẹ của DOM thực. Khi có sự thay đổi, Next.js sẽ cập nhật Virtual DOM, so sánh với DOM thực và chỉ thực hiện các thay đổi cần thiết, giúp cải thiện hiệu suất. [7]

3.2.3. Inngest – Nền tảng Điều phối Sự kiện

Inngest là một nền tảng điều phối sự kiện (Event Orchestration Platform) hiện đại, được thiết kế chuyên biệt cho các kiến trúc Serverless và Microservices. Khác với các hệ thống hàng đợi truyền thống (như RabbitMQ hay Kafka) đòi hỏi quản lý hạ tầng phức tạp, Inngest hoạt động dựa trên giao thức HTTP, cho phép các lập trình viên xây dựng các quy trình nghiệp vụ phức tạp, bền vững mà không cần quan tâm đến việc duy trì máy chủ hàng đợi. [10]

Inngest đóng vai trò là "cầu nối thông minh" giữa Frontend (Next.js) và bộ xử lý AI (Python Modal), đảm bảo quy trình review code diễn ra liền mạch và tin cậy.

Các đặc điểm nổi bật của Inngest:

- Thực thi bền vững: Inngest đảm bảo mọi sự kiện (Event) gửi đi đều được xử lý. Nếu Worker (Python) gặp lỗi hoặc bị timeout, Inngest sẽ tự động thử lại (Retry) theo cơ chế thông minh (Exponential backoff) mà không làm mất dữ liệu.
- Hỗ trợ đa ngôn ngữ: Inngest cho phép gửi sự kiện từ Node.js/TypeScript và xử lý sự kiện đó bằng Python SDK, giải quyết bài toán giao tiếp giữa hai môi trường công nghệ khác nhau trong dự án.

- Quy trình làm việc phức tạp: Hỗ trợ các tính năng điều phối nâng cao như: chờ đợi (sleep), gom nhóm sự kiện (batching), hoặc hủy bỏ tác vụ mà không cần viết code logic phức tạp.

Các thành phần chính của Inngest:

- Event (Sự kiện): Là tín hiệu kích hoạt quy trình, chứa dữ liệu cần xử lý (Payload). Ví dụ: sự kiện chứa thông tin về Pull Request cần đánh giá.
- Function (Hàm xử lý): Là các đoạn mã logic sẽ chạy khi nhận được sự kiện. Trong hệ thống này, các hàm Inngest được triển khai trên cả Next.js và Modal Python (để xử lý AI nặng). [10]
- Steps (Các bước): Chia nhỏ một quy trình dài thành các bước tuần tự. Nếu một bước bị lỗi, hệ thống có thể chạy lại từ bước đó thay vì chạy lại từ đầu, giúp tiết kiệm tài nguyên tính toán (đặc biệt quan trọng khi gọi API AI tốn kém). [10]

3.3. Tìm hiểu về bài toán tự động hóa đánh giá mã nguồn

Tự động hóa đánh giá mã nguồn là một lĩnh vực quan trọng và đang phát triển mạnh mẽ trong Kỹ thuật Phần mềm và Trí tuệ nhân tạo. Mục tiêu chính của bài toán này là phân tích các đoạn mã nguồn (source code) thay đổi trong các Pull Request (PR), từ đó tự động phát hiện lỗi logic, lỗ hổng bảo mật, vấn đề hiệu năng hoặc vi phạm quy chuẩn lập trình mà không cần sự can thiệp thủ công ban đầu của con người.

Ví dụ, với một đoạn mã thay đổi (Diff) trong PR, hệ thống sẽ "đọc hiểu" ngữ cảnh, so sánh với các quy chuẩn của dự án và kiến thức lập trình tổng quát để đưa ra các nhận xét (comments) như: "Biến này có thể gây lỗi NullPointerException", "Hàm này nên được tách nhỏ để dễ bảo trì", hoặc "Đoạn code này chưa xử lý ngoại lệ". Kết quả thu được là các lập trình viên nhận được phản hồi tức thì, giảm tải áp lực cho người review (reviewer) và tăng tốc độ phát triển phần mềm.

Trong thực tế, việc đọc và hiểu code chiếm tới 60-70% thời gian làm việc của một lập trình viên. Hàng ngày, hàng nghìn dòng code được đẩy lên các kho lưu trữ (repository) và cần được kiểm tra kỹ lưỡng. Do đó, mô phỏng quy trình tư duy của một Senior Developer để review code với sự trợ giúp của Mô hình Ngôn ngữ Lớn (LLM) là một trong những ứng dụng thực tiễn nhất của Generative AI hiện nay.

Các kỹ thuật đánh giá mã nguồn:

Có nhiều phương pháp tiếp cận khác nhau được ứng dụng trong việc tự động hóa review code. Các phương pháp được chia thành hai nhóm kỹ thuật chính:

3.3.1. Kỹ thuật truyền thống:

Đây là các phương pháp dựa trên các quy tắc cứng và phân tích cú pháp mã nguồn mà không cần chạy chương trình.

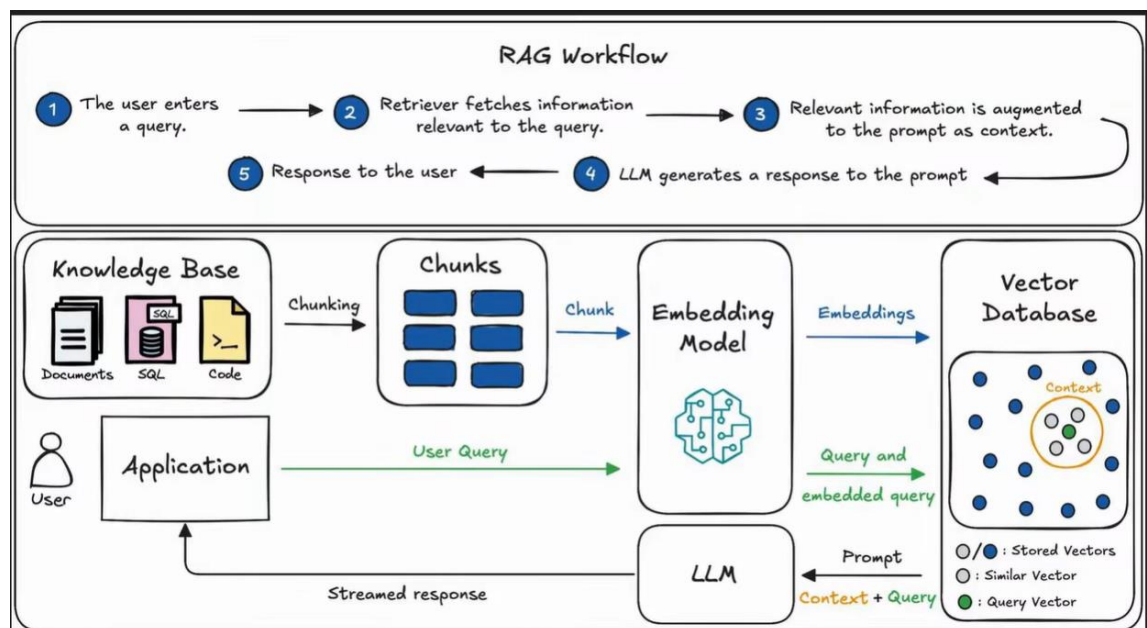
- Linter & Formatter: Các công cụ như ESLint, Prettier, Pylint tập trung vào việc kiểm tra lỗi cú pháp, định dạng code (indentation, spacing) và các lỗi phong cách cơ bản. Chúng hoạt động nhanh nhưng không hiểu được ngữ nghĩa sâu xa của code.
- Phân tích tĩnh (Static Application Security Testing - SAST): Các công cụ như SonarQube, CodeQL phân tích luồng dữ liệu (data flow) và luồng điều khiển (control flow) để tìm các lỗ hổng bảo mật phổ biến (SQL Injection, XSS) dựa trên các mẫu (patterns) đã biết trước.
- Kiểm tra dựa trên độ phức tạp (Complexity Metrics): Đo lường độ phức tạp Cyclomatic hoặc Cognitive Complexity để cảnh báo các hàm quá dài hoặc quá khó bảo trì.

3.3.2. Kỹ thuật trí tuệ nhân tạo tạo sinh dựa trên các mô hình ngôn ngữ lớn

Mô hình Ngôn ngữ Lớn (LLM – Large Language Models): Sử dụng các mô hình như GPT-5, Gemini, DeepSeek để đọc hiểu đoạn code như ngôn ngữ tự nhiên. LLM có khả năng phát hiện các lỗi logic, đề xuất cách tối ưu hóa thuật toán và thậm chí viết lại code tốt hơn.

Tạo sinh tăng cường truy xuất (RAG – Retrieval-Augmented Generation): Kỹ thuật kết hợp LLM với khả năng truy xuất dữ liệu từ cơ sở tri thức bên ngoài (như toàn bộ mã nguồn dự án), giúp AI hiểu được ngữ cảnh toàn cục (Global Context) mà các phương pháp truyền thống không làm được.

Agentic Workflow: Sử dụng các tác nhân AI có khả năng tự suy luận, lập kế hoạch kiểm tra để đưa ra đánh giá chính xác nhất.



Hình 1: Sơ đồ minh họa nguyên lý hoạt động của kiến trúc RAG

3.4. Ứng dụng RAG và Vector Database vào bài toán đánh giá mã nguồn

3.4.1. Giới thiệu về Retrieval-Augmented Generation (RAG)

RAG (Retrieval-Augmented Generation) là kỹ thuật tối ưu hóa đầu ra của các mô hình ngôn ngữ lớn bằng cách cung cấp thêm thông tin ngữ cảnh được truy xuất từ một nguồn dữ liệu đáng tin cậy bên ngoài trước khi thực hiện sinh câu trả lời. Trong bài toán Code Review, RAG giải quyết vấn đề "ảo giác" (hallucination) và thiếu hiểu biết về cấu trúc dự án của LLM. [1,9]

Thay vì chỉ dựa vào kiến thức đã được huấn luyện, hệ thống RAG sẽ tìm kiếm các đoạn code định nghĩa hàm, lớp hoặc các tài liệu kỹ thuật liên quan đến đoạn code đang được review, sau đó "nhồi" các thông tin này vào prompt để LLM có đủ dữ kiện đưa ra nhận xét chính xác. [1]

So sánh RAG và Fine-tuning trong bài toán Code Review:

Để tùy biến LLM cho một dự án cụ thể, có hai phương pháp chính là RAG và Fine-tuning. Dưới đây là bảng so sánh chi tiết để làm rõ lý do lựa chọn RAG cho hệ thống này:

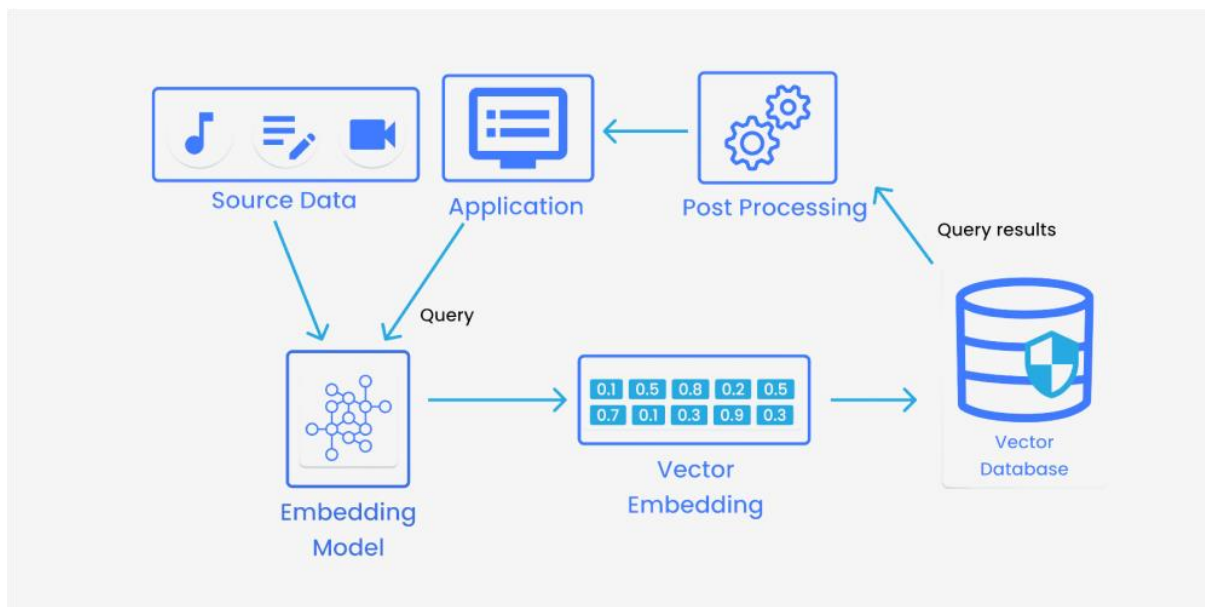
Tiêu chí	RAG (Retrieval-Augmented Generation)	Fine-tuning (Huấn luyện lại mô hình)
Cơ chế hoạt động	Truy xuất thông tin liên quan từ cơ sở dữ liệu (Database) và đưa vào ngữ cảnh (Context) của Prompt.	Huấn luyện lại trọng số (weights) của mô hình trên tập dữ liệu mới.
Tính cập nhật dữ liệu	Tức thì (Real-time): Khi code thay đổi, chỉ cần cập nhật lại Vector DB là AI biết ngay thông tin mới.	Chậm: Phải huấn luyện lại mô hình mỗi khi codebase thay đổi đáng kể, không phù hợp với dự án phát triển nhanh.
Chi phí triển khai	Thấp: Chỉ tốn chi phí lưu trữ vector và phí gọi API (Inference).	Rất cao: Tốn tài nguyên GPU đắt đỏ để train và host mô hình riêng.
Khả năng kiểm soát	Cao: Có thể biết chính xác AI đã tham khảo đoạn code nào để đưa ra nhận xét (Trích dẫn nguồn).	Thấp: Khó giải thích tại sao mô hình lại đưa ra kết quả đó.

Nguy cơ ảo giác	Thấp, vì AI bị ràng buộc bởi dữ liệu truy xuất được.	Vẫn có khả năng cao bị ảo giác nếu dữ liệu train không đủ lớn hoặc bị nhiễu.
Ứng dụng phù hợp	Cần độ chính xác cao về logic, tra cứu tài liệu, hiểu ngữ cảnh dự án động.	Cần thay đổi văn phong, học ngôn ngữ lập trình mới hoàn toàn chưa có trong pre-training.

Bảng 1: Bảng so sánh hai kiến trúc RAG và kỹ thuật Fine-tuning

Do đặc thù của mã nguồn dự án thay đổi liên tục theo từng commit, **RAG** là phương pháp tối ưu hơn hẳn so với Fine-tuning trong bài toán này.

3.4.2. Giới thiệu về Cơ sở dữ liệu Vector (Vector Database) - Pinecone



Hình 2: Nguyên lý vận hành của Pinecone (một loại vector database)

Vector Database là một loại cơ sở dữ liệu được thiết kế chuyên biệt để lưu trữ, quản lý và truy vấn dữ liệu dưới dạng vector nhiều chiều (embeddings). Trong hệ thống của bài toán đánh giá mã nguồn, mã nguồn (code) không được lưu dưới dạng văn bản thuần túy (text) mà được chuyển đổi thành các vector số học đại diện cho ý nghĩa ngữ nghĩa (semantic meaning) của đoạn code đó.

Pinecone là một trong những giải pháp Vector Database phổ biến và mạnh mẽ nhất hiện nay, cung cấp hạ tầng dưới dạng dịch vụ (Managed Service/Serverless).[2]

Cấu trúc và nguyên lý hoạt động:

- Embeddings: Mã nguồn (hàm, lớp, tài liệu) được đưa qua một mô hình Embedding (như Google Gemini Embedding hoặc OpenAI Ada) để chuyển thành vector (ví dụ: mảng n số thực). Các đoạn code có chức năng tương tự nhau sẽ có vị trí gần nhau trong không gian vector. [2]
- Indexing: Pinecone sử dụng các thuật toán chỉ mục tiên tiến (như HNSW - Hierarchical Navigable Small World) để tổ chức dữ liệu, cho phép tìm kiếm cực nhanh ngay cả với hàng tỷ vector.
- Similarity Search (Tìm kiếm tương đồng): Khi có một đoạn code cần review, hệ thống sẽ chuyển nó thành vector và truy vấn Pinecone để tìm ra "Top-K" đoạn code trong quá khứ có cấu trúc hoặc logic tương tự để làm tham chiếu. Sau đó truy vấn để tìm ra các đoạn code có cấu trúc hoặc logic tương tự trong không gian vector dựa trên độ tương đồng Cosine:

$$Similarity(A, B) = \frac{A \cdot B}{||A|| ||B||}$$

So sánh Pinecone với các giải pháp lưu trữ khác:

Tiêu chí	Pinecone (Chuyên dụng)	PostgreSQL + pgvector (Truyền thống)	Cơ sở dữ liệu quan hệ thường (MySQL/Postgres/SQL Server)
Loại dữ liệu	Vector Embeddings & Metadata	Dữ liệu quan hệ + Vector	Dữ liệu quan hệ, văn bản
Phương pháp tìm kiếm	Tìm kiếm ngữ nghĩa (Semantic Search): Tìm dựa trên ý nghĩa, ví dụ tìm code xử lý "đăng nhập" dù không có từ khóa "login".	Kết hợp ngữ nghĩa và từ khóa, nhưng hiệu năng vector kém hơn chuyên dụng.	Tìm kiếm từ khóa (Keyword Search): Chỉ tìm chính xác từ khóa (LIKE, Full-text search), không hiểu ngữ cảnh.
Hiệu năng quy mô lớn	Rất cao: Tối ưu hóa cho hàng tỷ vector, độ trễ thấp (mili-giây).	Trung bình: Hiệu năng giảm nhanh khi số lượng vector tăng lớn.	Không hỗ trợ vector search
Khả năng mở rộng	Serverless/Auto-scaling: Tự động mở rộng theo nhu cầu sử dụng mà không cần quản lý server.	Phụ thuộc vào cấu hình server database chính, khó mở rộng riêng phần vector.	Mở rộng theo chiều dọc (Vertical scaling).
Tính năng hỗ trợ AI	Tích hợp sẵn Hybrid Search (Kết hợp từ khóa + vector), lọc theo Metadata mạnh mẽ.	Hỗ trợ cơ bản, cần cài đặt extension và cấu hình phức tạp.	Không hỗ trợ các tác vụ AI.

Bảng 2: So sánh các loại database

3.4.3. Thiết lập môi trường lập trình

Để xây dựng hệ thống tự động hóa đánh giá mã nguồn (Code Review Automation), ta cần thiết lập môi trường Python serverless và cài đặt các thư viện chuyên dụng cho AI và xử lý dữ liệu.

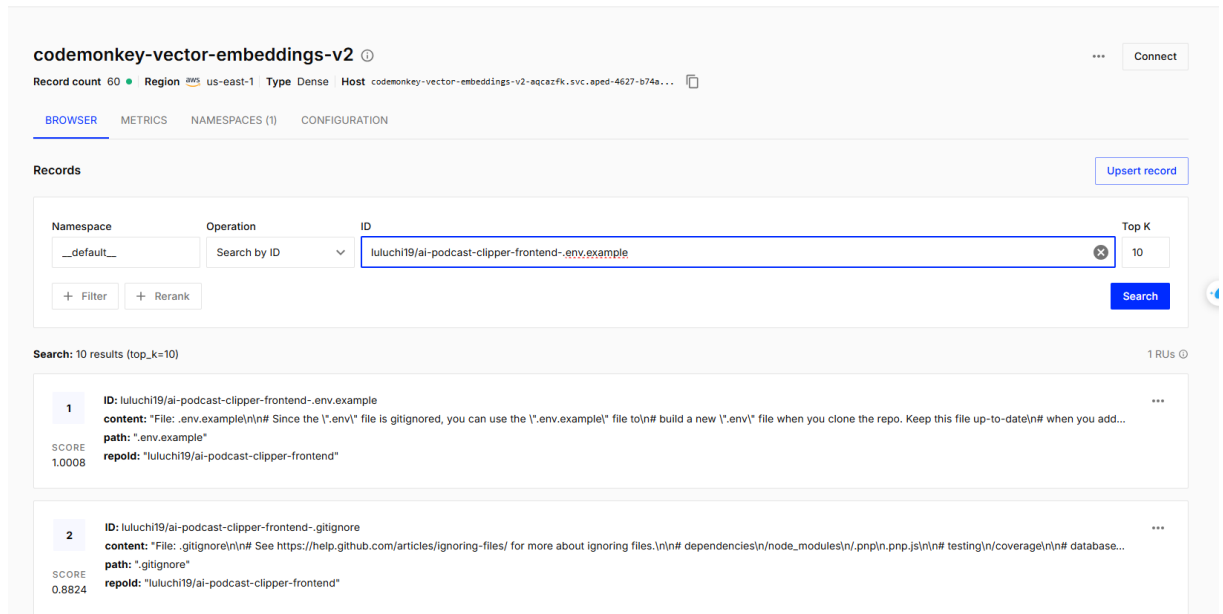
Các thư viện chính bao gồm:

- FastAPI & Uvicorn: Xây dựng API Server hiệu năng cao để giao tiếp giữa Client và Server.
- Modal: SDK để triển khai ứng dụng lên hạ tầng Serverless GPU/CPU.
- PydanticAI & Google-generativeai: Xây dựng AI Agent và kết nối với mô hình ngôn ngữ Gemini/DeepSeek.
- Pinecone-client: Tương tác với cơ sở dữ liệu Vector để lưu trữ và truy xuất kiến thức mã nguồn.
- PyGithub: Thư viện thao tác với GitHub API để lấy dữ liệu repository và đăng tải comment.

```
requirements.txt
1 fastapi==0.115.6
2 uvicorn[standard]==0.34.0
3
4 modal==0.67.38
5
6 pydantic==2.10.6
7 pydantic-ai==0.0.15
8
9 google-generativeai==0.8.5
10
11 pinecone-client==5.0.1
12
13 PyGithub==2.3.0
14
15 python-dotenv==1.0.1
16 httpx==0.28.1
```

3.4.4. Tải và cấu trúc luồng dữ liệu

Sau khi cài đặt các thư viện cần thiết, ta sẽ tiến hành quy trình "Indexing": tải mã nguồn thực tế từ GitHub, chia nhỏ (chunking) và vector hóa (embedding) để nạp vào Pinecone (Vector Database).



3.4.5. Xây dựng luồng xử lý RAG

RAG Agent bao gồm ba thành phần chính: **System Prompt**, **Context Retrieval** và **Structured Output**, hoạt động theo một luồng thống nhất.

Cấu trúc và nguyên lý hoạt động

- Khi nhận được một đoạn **Git Diff (Khác biệt sau khi thay đổi code)**, hệ thống xử lý theo các bước:
 - Chuyển nội dung Diff thành vector embedding.
 - Truy vấn Pinecone để lấy các đoạn mã liên quan trong repository.
 - Cung cấp Diff và Context cho AI Agent.
 - Agent thực hiện suy luận và sinh nhận xét ở dạng JSON có cấu trúc.
- Luồng xử lý tổng quát đơn giản:
 - Git Diff -> Embedding -> Pinecone Retrieval -> AI Agent (Reasoning + Generation) -> Structured JSON Output

rag.py > ...

```
1 from pydantic_ai import Agent
2 from schemas import ReviewResult
3
4 reviewer_agent = Agent(
5     model="google:gemin-2.5-flash",
6     system_prompt="""
7     Bạn là Senior Software Engineer.
8     Hãy review đoạn code diff dựa trên context được cung cấp.
9     Đánh giá các khía cạnh: logic, security, performance,
10    clean code và kiến trúc.
11    Trả về kết quả ở dạng JSON.
12    """,
13    result_type=ReviewResult
14 )
```

3.4.6. Triển khai API và xây dựng UI mô phỏng

3.4.6.1. Tạo API và Backend Serverless bằng FastAPI và Modal

Thay vì chạy server vật lý, toàn bộ backend được đóng gói thành các hàm Serverless trên nền tảng Modal, được kích hoạt thông qua FastAPI.

Ta khởi tạo đối tượng web_app là một FastAPI instance, định nghĩa các endpoint webhook để nhận sự kiện từ GitHub và connect để kết nối repository.

== CUDA ==

CUDA Version 12.4.0

Container image Copyright (c) 2016–2023, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

This container image and its contents are governed by the NVIDIA Deep Learning Container License.
This container image and its contents are governed by the NVIDIA Deep Learning Container License.
By pulling and using the container, you accept the terms and conditions of this license:
<https://developer.nvidia.com/ngc/nvidia-deep-learning-container-license>

A copy of this license is made available in this container at /NGC-DL-CONTAINER-LICENSE for your convenience.

Loading models

None

Processing: Generating semantic index from 1,500 document fragments for RAG system...

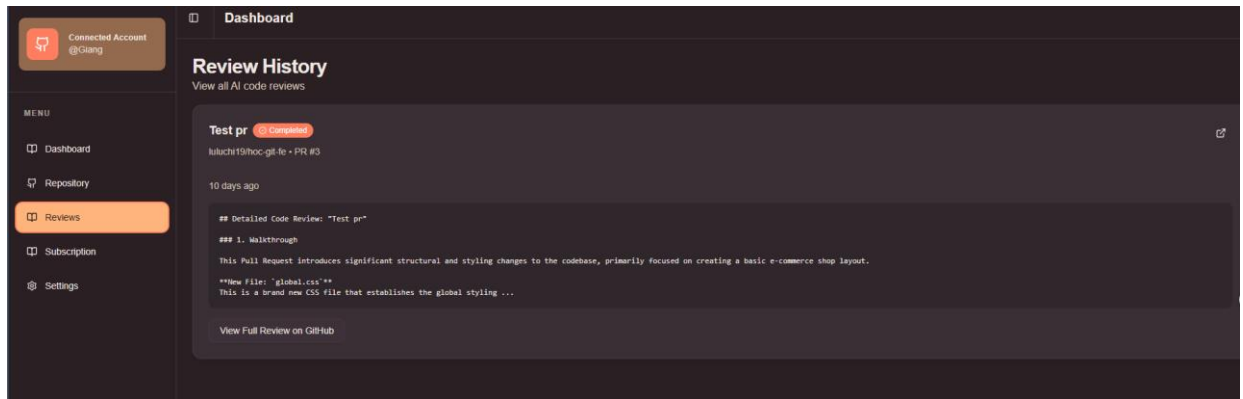
POST / -> 200 OK (duration: 4.34 s, execution: 47.3 ms)

Stopping app - local entrypoint completed.

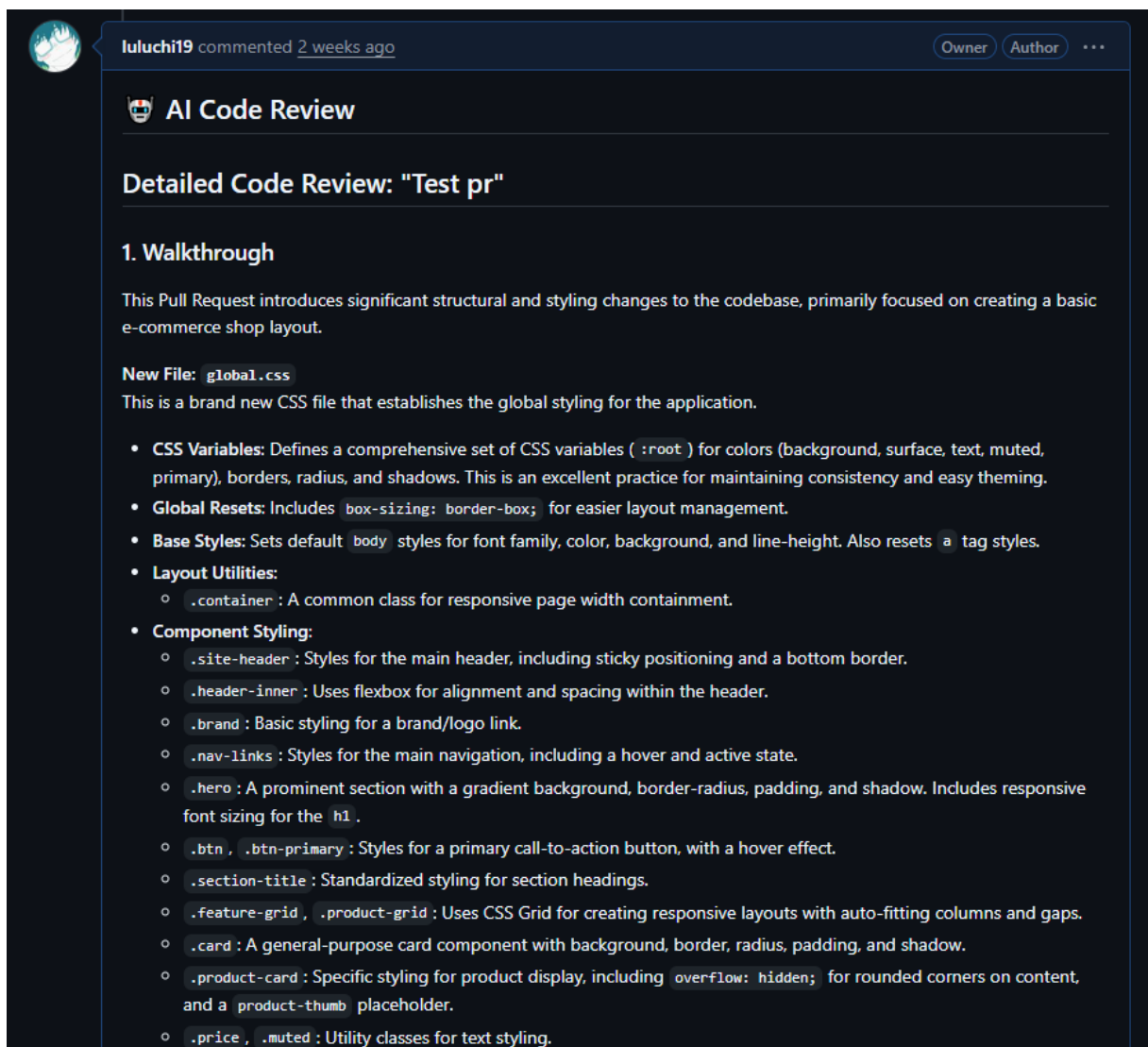
✓ App completed. View run at <https://modal.com/apps/luluchi19/main/ap-qRKqnemeQb1Cfw4vvv8iGpq>

3.4.6.2. Tạo giao diện người dùng bằng Next.js:

Tiếp theo, ta sẽ xây dựng giao diện mô phỏng bằng thư viện Next.js



Kích hoạt kết nối đến Github và trả về đánh giá sau khi có một Pull Request:



4. NHẬN XÉT VÀ ĐỊNH HƯỚNG

4.1. Nhận xét chung về đợt thực tập

Đợt thực tập diễn ra mang lại nhiều lợi ích cho bản thân em hiểu rõ hơn về quy trình thực tế trong công việc của mỗi vai trò khác nhau.

Việc được các anh chị hướng dẫn tại công ty về quy trình công việc giúp em thu hẹp khoảng cách giữa lí thuyết và thực tế.

4.2. Bài học kinh nghiệm rút ra

Kỹ năng tự học là yếu tố sống còn trong ngành công nghệ.

Việc thẳng thắn đặt vấn đề khi còn thắc mắc là yếu tố nhanh nhất để tăng hiệu quả làm việc, học tập cũng như giảm thiểu sai sót, tránh mắc sai lầm và tốn thời gian và tài nguyên khắc phục.

4.3. Định hướng nghề nghiệp và học tập sau thực tập

Dựa trên kinh nghiệm tích lũy được sau thời gian đi thực tập, tiếp tục hoàn thành mục tiêu ngắn hạn là đồ án tốt nghiệp

Tiếp tục theo đuổi con đường kỹ sư cầu nối BrSE theo lộ trình của công ty FPT Software.

TÀI LIỆU THAM KHẢO

- [1] Y. Gao, Y. Xiong, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2024. (Trang 2-10).
- [2] Pinecone, "What is a Vector Database & How Does it Work? Use Cases + Examples," 2026. [Online]. Available: <https://www.pinecone.io/learn/vector-database/>.
- [3] "PydanticAI: Model-driven agentic AI framework documentation," 2025. [Online]. Available: <https://ai.pydantic.dev>.
- [4] "FastAPI Framework Documentation - Interactive API reference," 2025. [Online]. Available: <https://fastapi.tiangolo.com>.
- [5] "PyGithub: A Python library to access the GitHub API v3," 2024. [Online]. Available: <https://pygithub.readthedocs.io>.
- [6] Microsoft, "TypeScript Documentation - Typed JavaScript at Any Scale," 2025.
- [7] Vercel, "Next.js 16 Documentation: App Router, Server Components and Actions," 2026.
- [8] Y. Zhang et al., "LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM," Beijing Institute of Technology, 2025. (Trang 1-6).
- [9] Shubham Kansal, "15 RAG Terms you Need to Know," Mouka, 2026. (Trang 1-16).
- [10] "Inngest Documentation", 2026 [Online].
Available: <https://www.inngest.com/docs>